

Read this!
All of this!
Carefully!

4 hours preparation time!

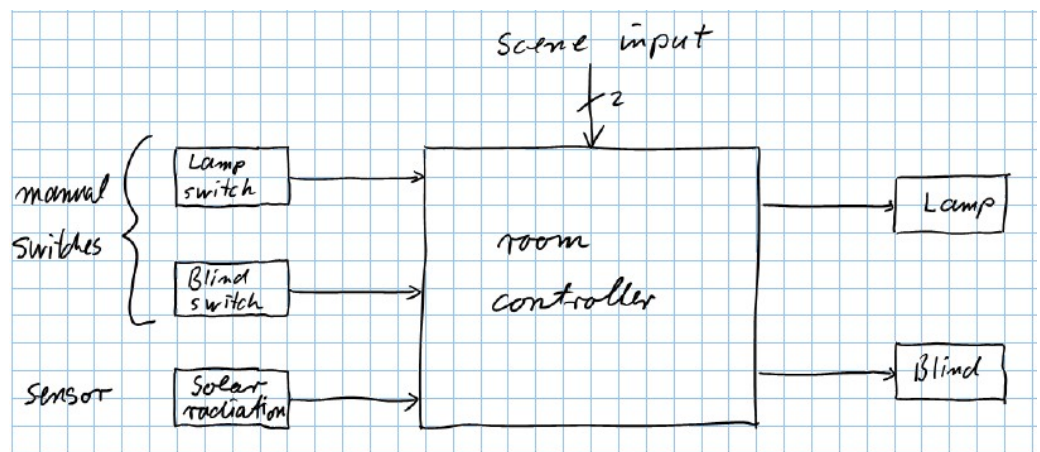
Digital Circuits

Lab session on Combinational Logic

Combinatorial circuit for a smart home living space.

Prof. Dr.-Ing. Peter Schulz

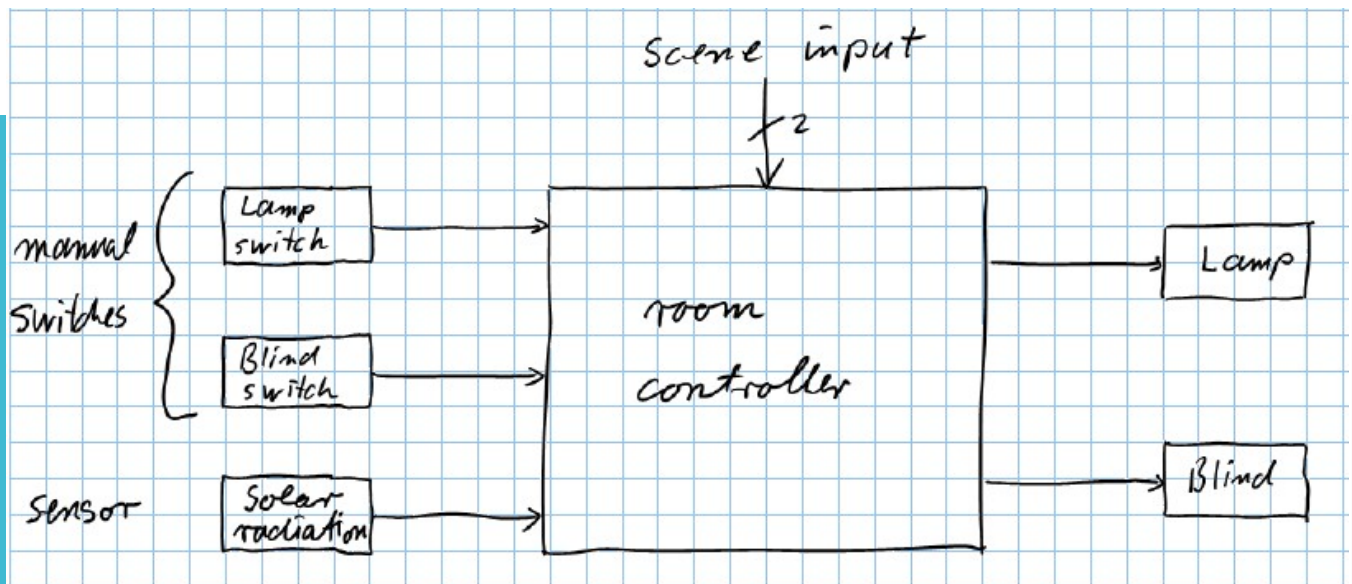
Revision 1.4, November 20th, 2024



Smart Home Logic

Objectives

- Design a room controller circuit.
- Build the circuit using SSI ICs.
- Implement the circuit using FPGA.



Requirements list

- The circuit has two scene inputs, a sensor input and two switching inputs for manual operation.
- The Scenes are "Standard", "Lunchtime", "Nighttime" and "Emergency"
 - The scenes are coded with a 2-bit bitvector:
 - "00" Standard, "01" Lunchtime, "10" Nighttime, "11" Emergency
- The sensor input indicates with a '1' that the solar radiation has exceeded a threshold value.
- The switching inputs are for manual operation of light and blind provided and connected with switches.
 - A closed switch is reported as '1'.
 - The corresponding switch must be closed for shading (blinds).
 - For light switched on, the associated switch must also be closed.
- The circuit has two outputs:
 - one for the light ('0': off, '1': on)
 - one for the blinds (shading, '0': blind down, '1': blind up).
- The function is specified as follows:
 - In the "Standard" scene, the logic only reacts to manual operation using a switch (light sensor is ignored).
 - In "Lunchtime" Scene manual operation is inhibited. If the sun is shining during the midday rest period (light sensor above the threshold value), the blinds should be closed.
 - At "Nighttime", the light is manually operated, and the blinds are closed (shaded).
 - At "Emergency", the light is permanently on, and the blinds are permanently open (no shading).

Preparation

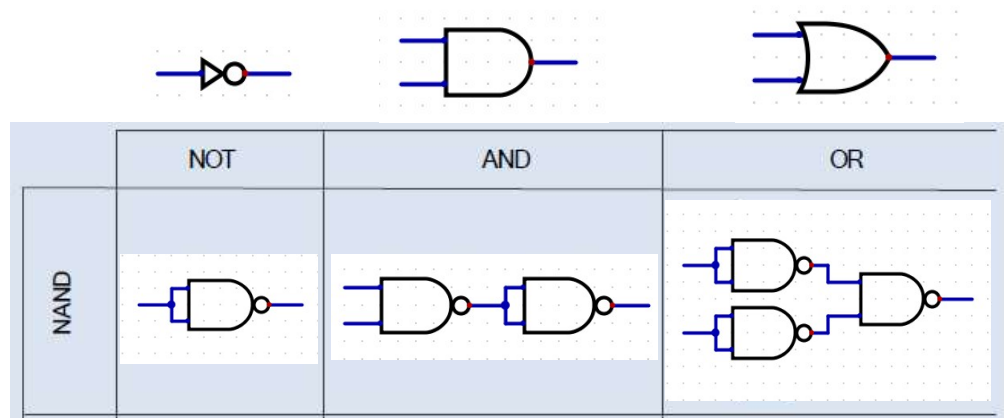


Fig. 1: Equivalent implementation for NOT, AND, OR with NAND only [1]

- Preparation Task 1 (Digital program)
 - Install the "Digital" program by H. Neemann (<https://github.com/hneemann/Digital>)
 - If not yet done run through the getting started tutorial. This can be accessed with the tool.
- Preparation Task 2 (Digital program)
 - Assign short but meaningful names to all input and output signals.
 - Design truth tables first and then a matching combinational circuit.
 - Use the program "Digital", to generate and verify the circuit (latter by simulation).
 - Print out:
 - truth tables
 - generated circuit
 - Boolean equations
 - Simulation result
 - Export your circuit to VHDL and save the source file on your USB memory stick for the lab session.
 - Generate the Circuit as a NAND only version.
 - Print out the NAND only version.
- Preparation Task 3 (EDAplayground)
 - Setup an environment on EDAplayground and enter your VHDL code from Preparation Task 2 (the first version, not the NAND version). If you don't know how, consult [2].
 - Set up a test plan with a reasonable number of test-cases derived from the requirements list. Each test case must have a unique identifier/number (like R1.1, R1.2, R2.1 etc., where the R1.x group stands for scene 0 test cases, R2.x group for scene 1 test cases etc.). Use a test-plan table as shown on the next page.
 - Create a Testbench for the room controller and perform a simulation. The testbench code sections should appear in test plan order and the sections must start with the requirements identifiers in comment lines.

Print out this plan, fill in all test cases and bring it to the lab. During lab check all tests. The original, checked test plan must be added to your report.

[illegible]

Execution in the Laboratory

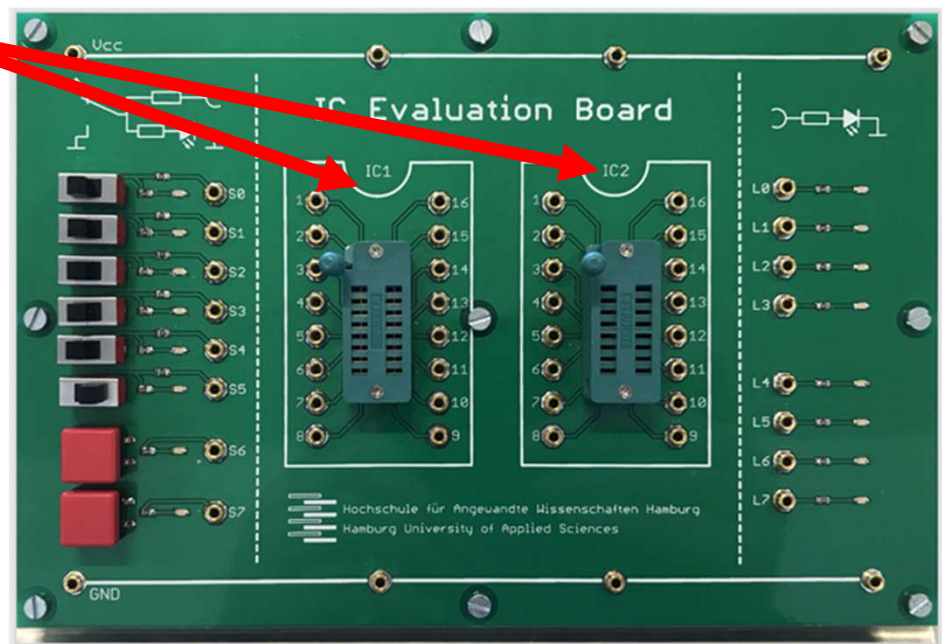


Fig. 2: Evaluation Board

- Lab Task 1
 - Build the "Lamp-part" of the room controller with 2-input NAND gates (Fig. 3) only using the Evaluation Board (Fig. 2).
 - While wiring the circuit: power supply is disconnected and off!
 - regard the orientation of the chip in the socket (markings!!!!)
 - regard that the socket is built for 16 pin ICs, but the 7400 has 14 pins only!
 - Make sure that you don't exceed the power supply values: GND = 0 V and the Vcc = 3.3 V
 - Assign switches and LEDs to the room controller's signals.
 - Test your circuit according for all "Lamp-related" test cases from your test plan.
 - Sketch the schematic of your circuit.
- Hint: Do not disassemble the circuit because you need it for further lab tasks.

regard these
markings!!!!

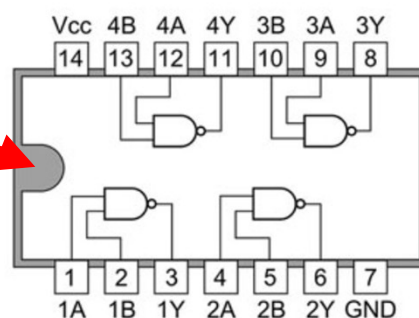


Fig. 3: Pin assignment of a 7400 device

picture source: <https://www.semiconductorforu.com/7400-series-integrated-circuits/>

Execution in the Laboratory

Preparing to use
the function
generator [1]

Prevent from damaging your circuit elements!

For stimulating the logic circuits with the function generator use the prepared terminated coaxial cables only!



Fig. 4: left: Coaxial signal cable with termination resistor and connection tips: **black** to GND, **red** to CMOS device logic input! right: termination resistor (don't disconnect)

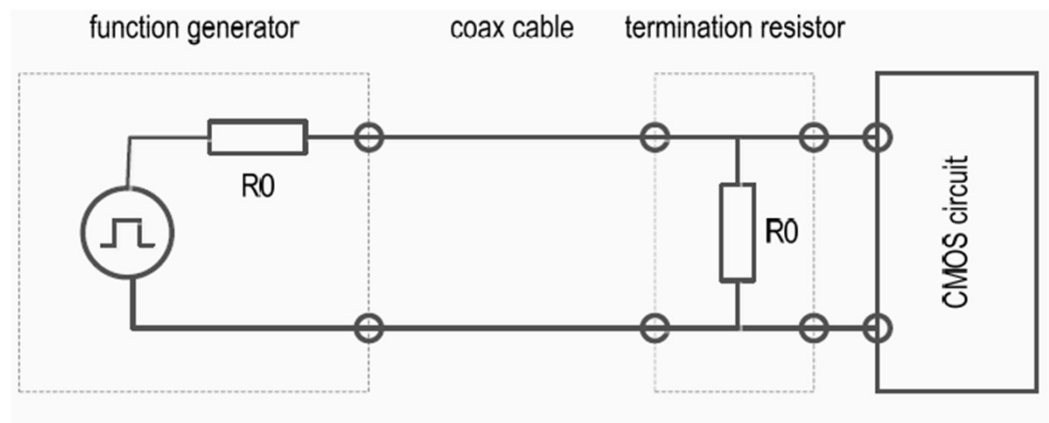


Fig. 5: Basic measurement setup using a function generator [1]

Attention: the voltage that we measure at the output of the function generator with an oscilloscope might not be the same as shown on the display of the function generator. Often it is two times higher! Why? Because the function generator was setup in a way that assumes the usage of a termination resistor (see below). Using a termination resistor is a common practice for higher frequency signals in order to reduce reflection on the long cable between function generator and our digital circuit. So, it gives us a cleaner signal to observe. The termination resistor has the same value as both the output resistor of the function generator and the characteristic impedance of the cable (in our case: 50Ω). The voltage divider created by output resistor of the function generator and termination resistor halves the input voltage at the input CMOS circuit.

Prevent from damaging your circuit elements!

Attention:

- No matter whether you use a termination resistor or not, always take a measurement to control the actual output voltage of the function generator at the point where you want to connect it to your digital circuit before connecting it to the circuit
- Apart from this, never apply negative voltages to CMOS inputs, this means you have to add an offset to the function generator output voltage.
- Pls. double check with the oscilloscope that you did everything correctly (input voltage to CMOS circuit does not exceed $V_{min}=0V$, $V_{max}=V_{cc}=5V$).

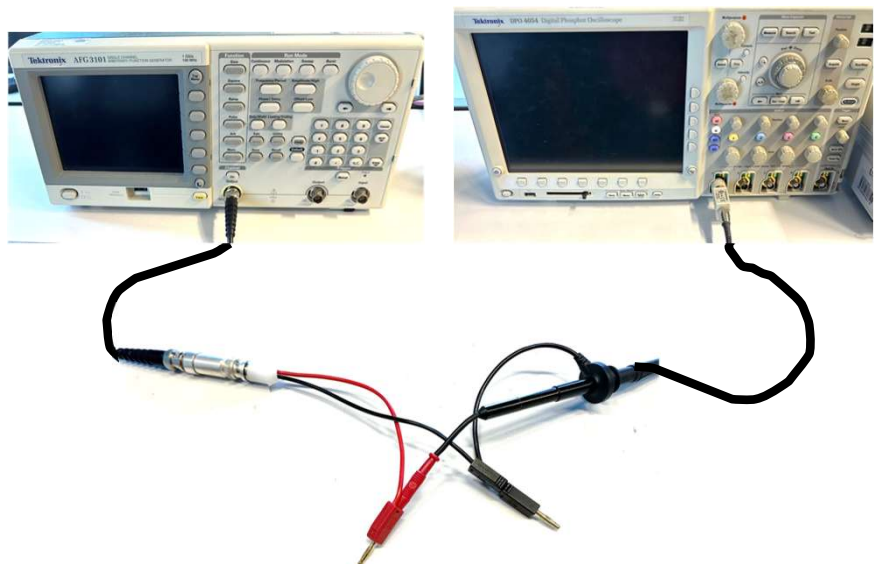


Fig. 6: How to check setup of a function generator

Execution in the Laboratory

Preparing to use the function generator [1]

- Lab Task 2.1
 - Connect the function generator output to oscilloscope channel one using a terminated cable (figure 6, no digital circuit involved!)
 - Perform adjustment of the function generator:
 - Square Wave
 - frequency: s.th. between 10 and 100 kHz
 - amplitude: $3.3 V_{pp}$ (peak-to-peak)
 - offset: output voltage alternates between 0 V and 3.3 V.
 - inspect the function generator output signal carefully on the oscilloscope

Execution in the Laboratory

instruments set up

Fig.7 : Oscilloscope probe prepared for 8th floor labs. **Black:** to GND, **red:** to CMOS device logic output.

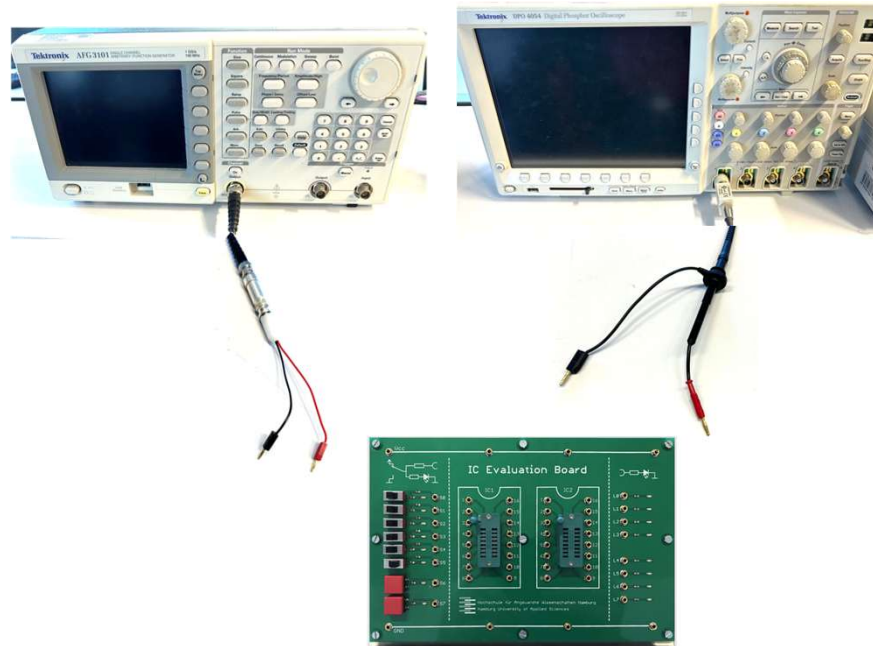


Fig. 8: basic measurement setup for task 2.2 and following tasks (probe setting not shown)

- Lab Task 2.2
 - Connect the function generator output to the Lamp Switch input of your circuit.
 - Connect the oscilloscope:
 - to channel 1 the function generator signal
 - to channel 2 the Lamp output of your circuit.
 - Perform a measurement for the propagation delay of this part of the circuit. (regard definition of propagation delay as explained with Fig. 9)
 - Sketch the circuit part involved in the measurement.
 - Take a screenshot of the oscilloscope and save it for your report.

Execution in the Laboratory

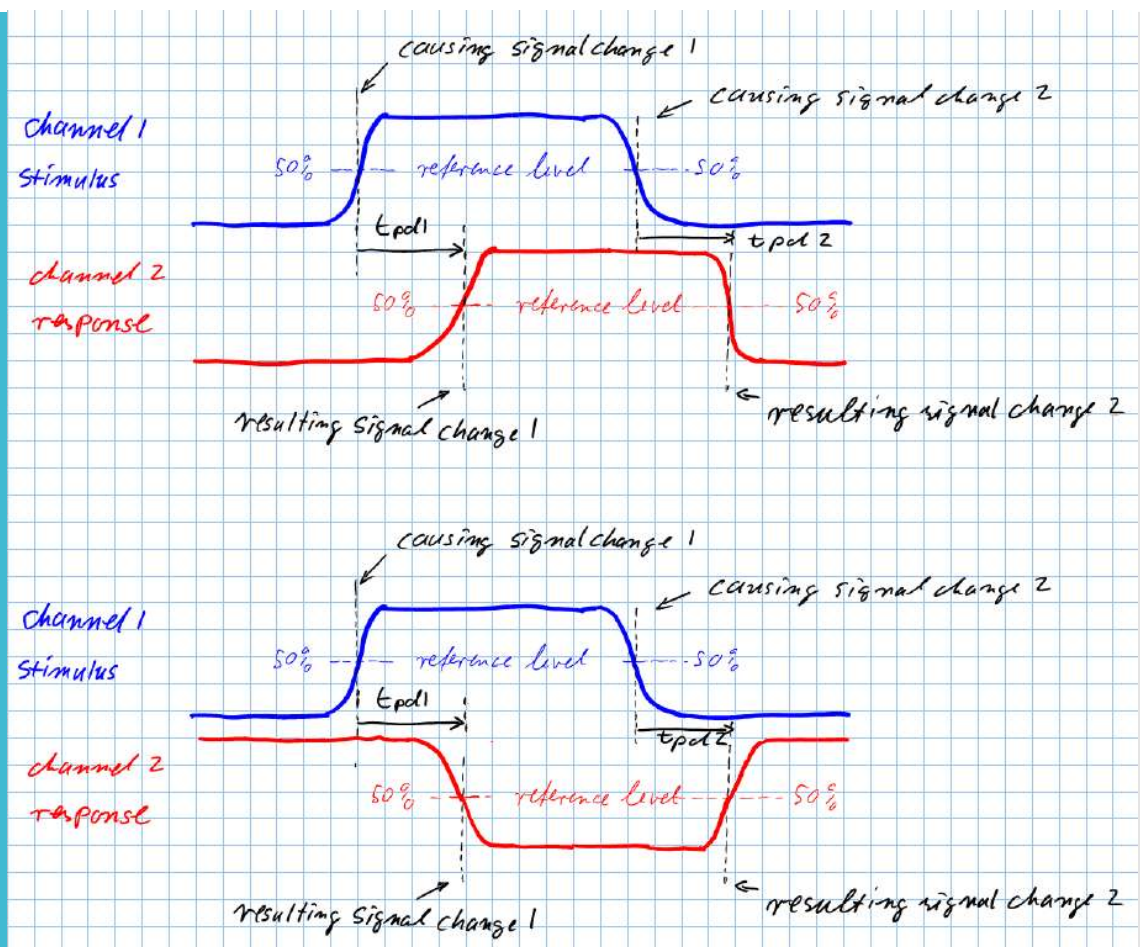


Fig. 9: Measurement of propagation delay t_{pd}

- Lab Task 2.3
 - Disassemble your circuit.
 - Build a test circuit with only one NAND gate involved as a NOT gate.
 - Perform a measurement for the propagation delay of this part of the circuit.
 - Sketch the circuit part involved in the measurement.
 - Take a screenshot of the oscilloscope and save it for your report.
- Report: compare and discuss results of lab tasks 2.2 and 2.3

Execution in the Laboratory

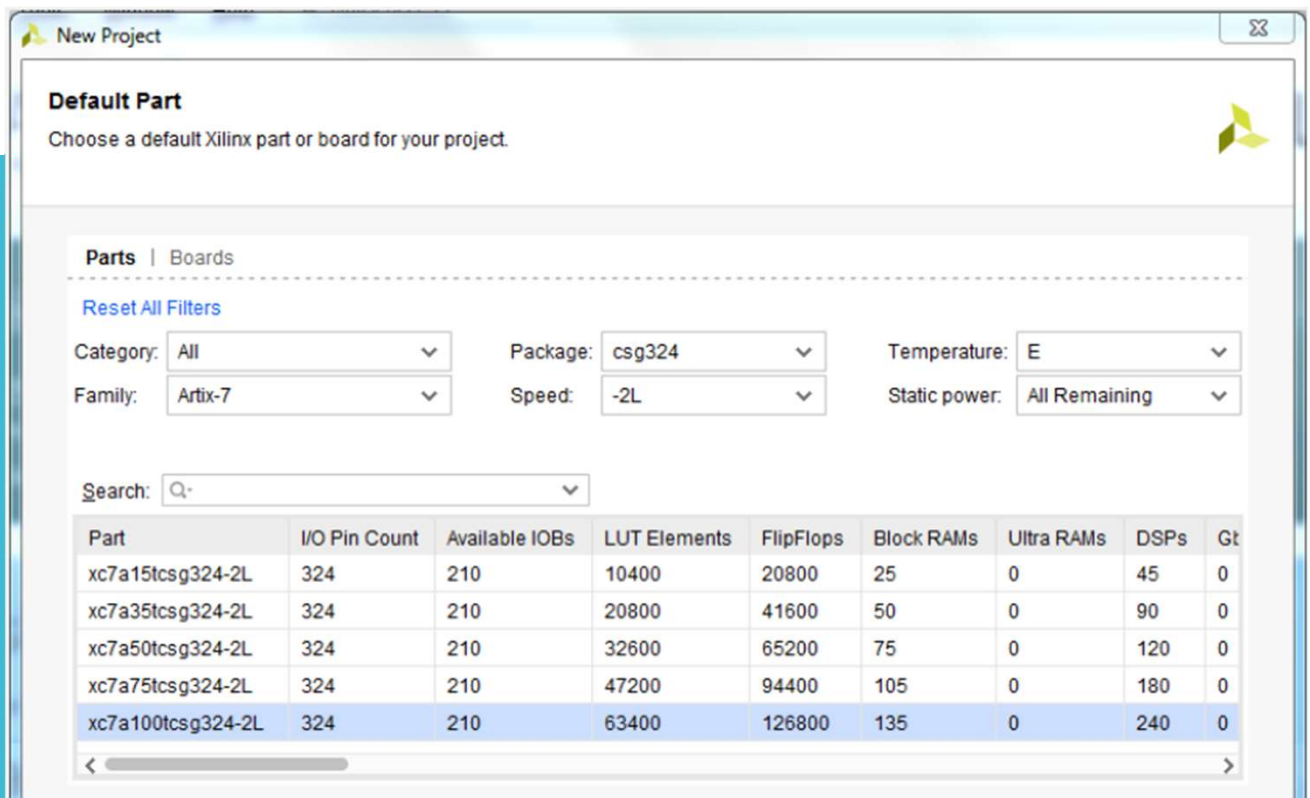
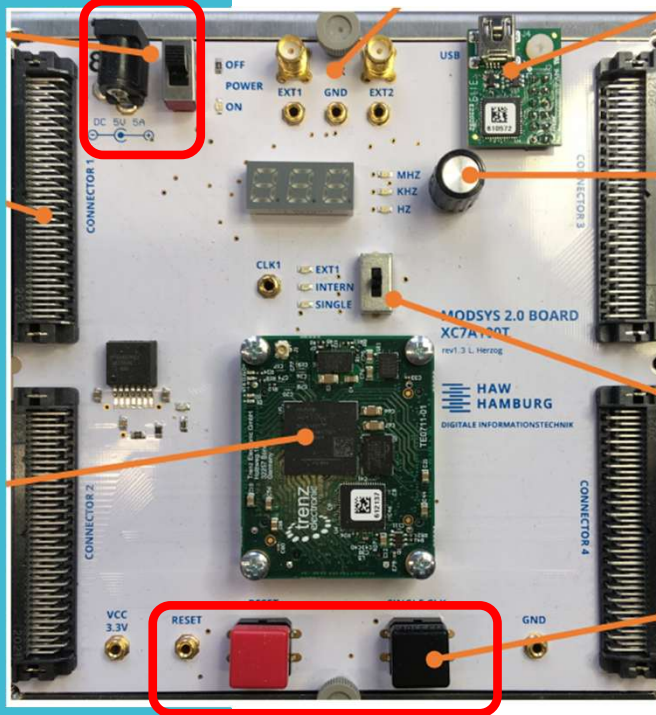


Fig. 10: Vivado project settings for lab sessions

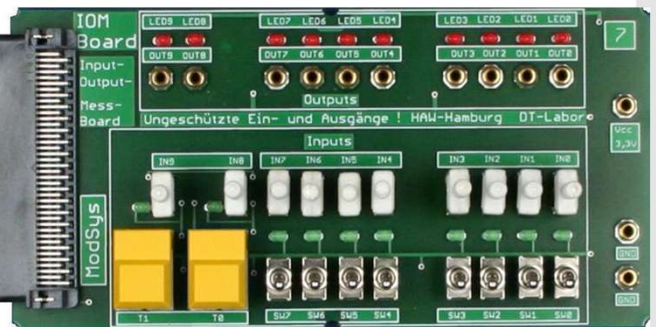
- Lab Task 3.1
 - Create a new RTL project in Vivado.
 - Add your room controller VHDL file which was exported from the Digital program during preparation. (The design file only, not a test-bench)
 - I/O specifications
 - Assign IOM board (Fig. 11) switches and LEDs so that you can perform tests on your room controller logic.
 - Add an IOM constraints file (xdc-file on Moodle). Select connector 4 version.
 - Edit the xdc file so that it fits to the signal names in your VHDL entity.
 - Although VHDL is case-insensitive, the XDC files are not. The signal names in the XDC file must look the same as in the entity.
 - Overwrite original names of switches or LEDs you want to use with your entity port names.
 - Comment out all other lines with IOM elements you don't use. Comment symbol is : #
 - The pin numbers in the XDC file must not be changed.
- Lab Task 3.2
 - Perform the Design Flow **without simulation**. (Synthesis, Implementation, Bit-Stream-Generation)
 - Inspect schematic files if they show reasonable results.
- Lab Task 3.3
 - Download the generated Bit-Stream to the FPGA Board and perform the test according to your test plan.

power connector and power switch,
don't detach the power cable!

FPGA-Board setup



clk and resetn, not used in this lab



IOM-Board on connector 4, use:
Adder_IOM_Conn_4.xdc

Execution in the Laboratory

Fig. 11: IOM Board for Stimuli generation and result inspection [1]
connected to the FPGA main unit

References

- [1] L. Leutelt, T. Link, D. Palme "LAB Digital Circuits", HAW Hamburg Teaching Material
- [2] P. Schulz, "EDApayground Tutorial"